

Kafka Complete Learning Lab

A comprehensive, hands-on Kafka learning environment with Docker, Go examples, and production-ready patterns.

☐ Project Structure

```
kafka-lab/
├─ README.md                # This file
├─ docker-compose-single.yml # Single broker setup
├─ docker-compose-cluster.yml # 3-broker cluster setup
├─ Makefile                 # Convenient commands
├─ go.mod                   # Go module definition
├─ go.sum                   # Go dependencies
├─
├─ docs/
│   ├─ kafka-reference-guide.md # Complete Kafka reference
│   ├─ cli-cheatsheet.md        # CLI commands quick reference
│   └─ troubleshooting.md       # Common issues and solutions
│
├─ examples/                # Go code examples
│   ├─ 01-simple-producer.go
│   ├─ 02-simple-consumer.go
│   ├─ 03-consumer-group.go
│   ├─ 04-dlq-pattern.go
│   ├─ 05-retry-pattern.go
│   ├─ 06-idempotent-consumer.go
│   ├─ 07-parallel-consumer.go
│   ├─ 08-request-reply.go
│   ├─ 09-performance-test.go
│   └─ 10-monitoring.go
│
├─ scripts/                 # Helper scripts
│   ├─ setup.sh              # Initial setup
│   ├─ create-topics.sh      # Create example topics
│   ├─ test-producer.sh      # Test producer
│   ├─ test-consumer.sh      # Test consumer
│   ├─ produce-json.sh       # Produce JSON messages
│   ├─ consume-json.sh       # Consume JSON messages
│   ├─ check-lag.sh          # Monitor consumer lag
│   ├─ reset-offsets.sh      # Reset offsets
│   ├─ topic-info.sh         # Get topic details
│   ├─ broker-health.sh      # Check broker health
│   ├─ performance-test.sh   # Run performance tests
│   ├─ simulate-failure.sh   # Simulate broker failure
│   ├─ restore-broker.sh     # Restore failed broker
│   ├─ logs.sh               # View logs
│   ├─ cleanup.sh            # Cleanup everything
│   └─ quick-test.sh         # Quick end-to-end test
│
```

```
└─ labs/                                     # Step-by-step lab exercises
    ├── lab-01-basics.md
    ├── lab-02-producers.md
    ├── lab-03-consumers.md
    ├── lab-04-consumer-groups.md
    ├── lab-05-patterns.md
    ├── lab-06-fault-tolerance.md
    └─ lab-07-performance.md
```

Quick Start

Prerequisites

- Docker 20.x+ and Docker Compose
- Go 1.24.0+
- 8GB RAM recommended
- 10GB free disk space

Option 1: Using Make (Recommended)

```
# Clone or create the project
mkdir kafka-lab && cd kafka-lab

# Download all files from the artifacts provided

# Make scripts executable
chmod +x scripts/*.sh

# Start single broker
make start-single

# Or start 3-broker cluster
make start-cluster

# Create example topics
make topics

# Run quick test
make test

# Open Kafka UI
open http://localhost:8080
```

Option 2: Manual Setup

```
# Start Kafka
docker-compose -f docker-compose-single.yml up -d

# Wait for startup
sleep 30

# Create topics
./scripts/create-topics.sh

# Initialize Go module
cd examples
go mod init kafka-lab
go get github.com/segmentio/kafka-go@v0.4.48

# Run example
go run 01-simple-producer.go
```

☐ Learning Path

Day 1: Basics (2-3 hours)

1. Read: `docs/kafka-reference-guide.md` - Core Concepts section
2. Do: `labs/lab-01-basics.md` - Setup and CLI basics
3. Run: `examples/01-simple-producer.go` and `02-simple-consumer.go`
4. Practice: CLI commands from `docs/cli-cheatsheet.md`

Day 2: Producers (2-3 hours)

1. Read: Producers section in reference guide
2. Do: `labs/lab-02-producers.md`
3. Run: Examples 01-04 (producers with different configs)
4. Experiment: Change acks, batch sizes, compression

Day 3: Consumers (2-3 hours)

1. Read: Consumers and Consumer Groups sections
2. Do: `labs/lab-03-consumers.md` and `lab-04-consumer-groups.md`
3. Run: Examples 02, 03, 06, 07
4. Practice: Monitor lag with `./scripts/check-lag.sh`

Day 4: Patterns (3-4 hours)

1. Read: Message Queue Patterns section
2. Do: `labs/lab-05-patterns.md`
3. Run: DLQ, retry, request-reply examples (04, 05, 08)
4. Build: Your own pattern combining learnings

Day 5: Production Readiness (3-4 hours)

1. Read: Replication, Performance, Best Practices sections
2. Do: `labs/lab-06-fault-tolerance.md` and `lab-07-performance.md`
3. Run: Performance tests and failure simulations
4. Review: Troubleshooting guide

☐ Available Commands

Make Commands

```
make help          # Show all commands
make start-single  # Start single broker
make start-cluster # Start 3-broker cluster
make stop          # Stop all containers
make clean         # Remove everything
make test          # Quick test
make topics        # Create topics
make health        # Check health
make logs          # View logs
```

Script Commands

```
# Testing
./scripts/quick-test.sh
./scripts/test-producer.sh <topic> <count>
./scripts/test-consumer.sh <topic> <group>
./scripts/produce-json.sh <topic> <count>
./scripts/performance-test.sh

# Monitoring
./scripts/check-lag.sh <group>
./scripts/topic-info.sh <topic>
./scripts/broker-health.sh
./scripts/logs.sh <container> <lines>

# Management
./scripts/reset-offsets.sh <group> <topic> [earliest|latest]
./scripts/simulate-failure.sh
./scripts/restore-broker.sh <broker>
./scripts/cleanup.sh
```

Documentation

Complete Reference Guide

Location: See artifact "Complete Kafka Reference Guide"

Covers:

- All core concepts in depth
- Architecture and internals

- Producer and consumer deep dives
- Message queue patterns
- Complete CLI commands cheatsheet
- Performance tuning
- Best practices
- Troubleshooting

Go Code Examples

Location: See artifact "Kafka Go Code Examples"

15 complete examples:

1. Simple Producer
2. Durable Producer (acks=all, idempotence)
3. Batch Producer (high throughput)
4. Async Producer
5. Simple Consumer
6. Consumer Group with Manual Commit
7. Consumer Group with Auto Commit
8. Multi-Topic Consumer
9. Graceful Shutdown
10. Parallel Processing
11. Dead Letter Queue Pattern
12. Retry Pattern
13. Request-Reply Pattern
14. Idempotent Consumer
15. Monitoring and Metrics

Hands-On Lab Guide

Location: See artifact "Kafka Hands-On Lab - Step-by-Step Guide"

7 comprehensive labs:

- Part 1: Setup and Basic Operations
- Part 2: Go Producer and Consumer
- Part 3: Advanced Patterns
- Part 4: Multi-Broker Cluster
- Part 5: Performance Testing
- Part 6: Monitoring and Debugging
- Part 7: Cleanup and Troubleshooting

Helper Scripts

Location: See artifact "Kafka Lab Helper Scripts"

20+ production-ready scripts for:

- Setup and initialization
- Testing and debugging
- Monitoring and health checks
- Performance testing
- Failure simulation
- Data management

Configuration

Single Broker Setup

- **Use for:** Development, learning, quick testing
- **Ports:**
 - Kafka: 9092
 - Zookeeper: 2181
 - Kafka UI: 8080
- **Resources:** ~2GB RAM, 2GB disk

3-Broker Cluster

- **Use for:** Production-like testing, fault tolerance learning
- **Ports:**
 - Kafka 1: 9092
 - Kafka 2: 9093
 - Kafka 3: 9094
 - Zookeeper: 2181
 - Kafka UI: 8080
- **Resources:** ~6GB RAM, 6GB disk

Labs and Exercises

Lab 1: Kafka Basics

- Create, describe, delete topics
- Produce and consume via CLI
- Understand partitions and offsets
- Work with consumer groups

Lab 2: Go Producers

- Simple fire-and-forget producer
- Synchronous producer with error handling
- Batch producer for high throughput
- Async producer with callbacks
- Compare acks levels

Lab 3: Go Consumers

- Simple consumer from specific partition
- Consumer group with load balancing
- Manual vs auto commit
- Handle rebalancing
- Monitor consumer lag

Lab 4: Consumer Group Scaling

- Start with 1 consumer
- Scale to match partition count
- Observe load distribution
- Test with more consumers than partitions
- Monitor rebalancing

Lab 5: Message Queue Patterns

- Point-to-point (queue)
- Publish-subscribe (fan-out)
- Dead letter queue
- Retry with exponential backoff
- Request-reply

Lab 6: Fault Tolerance

- Create replicated topics
- Simulate broker failure
- Observe leader election
- Check under-replicated partitions
- Restore failed broker
- Test data durability

Lab 7: Performance Tuning

- Producer throughput test
- Consumer throughput test
- Test different configurations

- Monitor metrics
- Optimize for your use case

□ Troubleshooting

Common Issues

Kafka not starting

```
# Check logs
make logs

# Check disk space
df -h

# Cleanup and restart
make clean
make start-single
```

Connection refused

```
# Wait for startup (30 seconds)
sleep 30

# Check containers
docker ps

# Restart
make stop && make start-single
```

Consumer not receiving messages

```
# Check topic exists
docker exec kafka kafka-topics --bootstrap-server localhost:9092 --list

# Check messages in topic
./scripts/topic-info.sh <topic>

# Check consumer group
./scripts/check-lag.sh <group>

# Reset offsets if needed
./scripts/reset-offsets.sh <group> <topic> earliest
```

High consumer lag

```
# Check lag
./scripts/check-lag.sh <group>

# Add more consumers (up to partition count)
# Or optimize processing logic

# Monitor in real-time
watch -n 2 './scripts/check-lag.sh <group>'
```

Out of memory

```
# Check Docker resources
docker stats

# Reduce batch sizes in producer
# Reduce max.poll.records in consumer
# Increase Docker memory allocation
```

For more troubleshooting: See "Troubleshooting Guide" section in the Complete Reference Guide

Monitoring

Kafka UI (Web Interface)

- **URL:** <http://localhost:8080>
- **Features:**
 - Browse topics and messages

- View consumer groups and lag
- Monitor broker health
- Manage configurations

CLI Monitoring

```
# Broker health
./scripts/broker-health.sh

# Consumer lag
./scripts/check-lag.sh <group>

# Topic details
./scripts/topic-info.sh <topic>

# Live logs
./scripts/logs.sh kafka

# Continuous health monitoring
watch -n 5 './scripts/broker-health.sh'
```

Go Application Metrics

Run the monitoring example:

```
cd examples
go run 10-monitoring.go
```

Shows:

- Messages processed
- Processing errors
- Average latency
- Queue length and capacity
- Current offset and lag

Real-World Use Cases

Use Case 1: Event-Driven Microservices

Pattern: Publish-Subscribe

```
Order Service → orders topic → [Payment Service, Inventory Service, Email Service]
```

Example: Run `examples/03-consumer-group.go` with multiple services

Use Case 2: Log Aggregation

Pattern: Point-to-Point Queue

```
Multiple Apps → logs topic → Log Processor (Consumer Group)
```

Example: Use `examples/07-parallel-consumer.go`

Use Case 3: Real-Time Analytics

Pattern: Stream Processing

```
Events → analytics topic → Real-time Processor → Metrics Dashboard
```

Example: Use `examples/09-performance-test.go`

Use Case 4: Task Queue

Pattern: Message Queue with DLQ

```
Tasks → task-queue → Workers → [Success | DLQ]
```

Example: Run `examples/04-dlq-pattern.go`

Use Case 5: CQRS Pattern

Pattern: Event Sourcing

```
Commands → events topic → Read Model Updater → Query Database
```

Example: Combine producer and consumer examples

☐ Advanced Topics

Transactions

```
// Enable transactional producer
writer := &kafka.Writer{
    Addr:      kafka.TCP("localhost:9092"),
    Topic:     "orders",
    RequiredAcks: kafka.RequireAll,
    // Note: segmentio/kafka-go doesn't fully support transactions yet
    // For full transaction support, consider confluent-kafka-go
}
```

Exactly-Once Semantics

Achieved through:

1. Producer idempotence (prevents duplicates)
2. Transactional writes
3. Read committed isolation level

See reference guide for detailed explanation.

Custom Partitioners

```
type CustomPartitioner struct{}

func (p *CustomPartitioner) Balance(msg kafka.Message, partitions ...int) int {
    // Custom logic
    // Example: Hash by user ID for user-based partitioning
    userID := extractUserID(msg.Key)
    return int(hash(userID)) % len(partitions)
}
```

Schema Registry Integration

For production systems, consider:

- Avro schemas
- Schema Registry (Confluent)
- Schema evolution and compatibility

Security

Enable in docker-compose:

```
environment:
  KAFKA_SECURITY_PROTOCOL: SASL_SSL
  KAFKA_SASL_MECHANISM: PLAIN
  KAFKA_SSL_KEYSTORE_LOCATION: /path/to/keystore
```

☐ Performance Benchmarks

Typical Single Broker Performance

- **Throughput:** 50-200 MB/s
- **Latency:** 2-10ms (p99)
- **Messages/sec:** 50K-200K (1KB messages)

With 3-Broker Cluster

- **Throughput:** 100-500 MB/s
- **Latency:** 5-15ms (p99)
- **Messages/sec:** 100K-500K (1KB messages)

Run your own tests:

```
./scripts/performance-test.sh
```

☐ Migration from Other Systems

From RabbitMQ

Key Differences:

- Kafka: Log-based, persistent, replay possible
- RabbitMQ: Queue-based, message deleted after consumption

Migration Strategy:

1. Keep both systems initially
2. Dual-write to both
3. Migrate consumers gradually
4. Switch off old system

From AWS SQS/SNS

Advantages of Kafka:

- Lower cost at scale
- Better throughput
- Message replay capability
- On-premises option

Considerations:

- Need to manage infrastructure (or use managed Kafka)
- Different guarantee models

☐ Deployment Options

Local Development

```
make start-single
```

Docker Compose (This Lab)

```
make start-cluster
```

Kubernetes

Use Strimzi operator or Confluent Operator

Managed Services

- **Confluent Cloud:** Fully managed
- **AWS MSK:** Managed Kafka on AWS
- **Azure Event Hubs:** Kafka-compatible
- **Aiven:** Multi-cloud managed Kafka

☐ Testing Strategies

Unit Testing

```
// Mock Kafka for unit tests
type MockWriter struct {
    messages []kafka.Message
}

func (m *MockWriter) WriteMessages(ctx context.Context, msgs ...kafka.Message) error {
    m.messages = append(m.messages, msgs...)
    return nil
}
```

Integration Testing

Use the Docker setup:

```
# Start test environment
make start-single

# Run tests
go test ./...

# Cleanup
make clean
```

Load Testing

```
# Producer load test
./scripts/performance-test.sh

# Consumer load test with multiple instances
for i in {1..5}; do
    go run examples/03-consumer-group.go &
done
```

☐ Learning Resources

Official Documentation

- **Apache Kafka:** <https://kafka.apache.org/documentation/> (<https://kafka.apache.org/documentation/>)
- **Kafka Improvement Proposals:**
<https://cwiki.apache.org/confluence/display/KAFKA/Kafka+Improvement+Proposals>

(<https://cwiki.apache.org/confluence/display/KAFKA/Kafka+Improvement+Proposals>).

Go Kafka Libraries

- **segmentio/kafka-go**: <https://github.com/segmentio/kafka-go> (<https://github.com/segmentio/kafka-go>) (This lab)
- **confluent-kafka-go**: <https://github.com/confluentinc/confluent-kafka-go> (<https://github.com/confluentinc/confluent-kafka-go>) (More features)
- **Sarama**: <https://github.com/Shopify/sarama> (<https://github.com/Shopify/sarama>) (Popular alternative)

Books

- "Kafka: The Definitive Guide" by Neha Narkhede
- "Designing Data-Intensive Applications" by Martin Kleppmann

Online Courses

- Confluent Kafka courses (free tier available)
- Udemy Kafka courses
- LinkedIn Learning

Community

- Kafka Users mailing list
- Stack Overflow (tag: apache-kafka)
- Confluent Community Slack

☐ Contributing

This lab is designed for learning. Feel free to:

- Add more examples
- Improve documentation
- Create additional lab exercises
- Share your use cases

☐ Cheat Sheet

Quick Commands

```
# Start/Stop
make start-single / make stop

# Create topic
docker exec kafka kafka-topics --bootstrap-server localhost:9092 \
  --create --topic TOPIC --partitions 3 --replication-factor 1

# List topics
docker exec kafka kafka-topics --bootstrap-server localhost:9092 --list

# Describe topic
docker exec kafka kafka-topics --bootstrap-server localhost:9092 \
  --describe --topic TOPIC

# Console producer
docker exec -it kafka kafka-console-producer \
  --bootstrap-server localhost:9092 --topic TOPIC

# Console consumer
docker exec -it kafka kafka-console-consumer \
  --bootstrap-server localhost:9092 --topic TOPIC --from-beginning

# Consumer groups
docker exec kafka kafka-consumer-groups \
  --bootstrap-server localhost:9092 --list

# Describe group
docker exec kafka kafka-consumer-groups \
  --bootstrap-server localhost:9092 --group GROUP --describe

# Reset offsets
docker exec kafka kafka-consumer-groups \
  --bootstrap-server localhost:9092 --group GROUP --topic TOPIC \
  --reset-offsets --to-earliest --execute
```

Go Producer Template

```

writer := &kafka.Writer{
    Addr:      kafka.TCP("localhost:9092"),
    Topic:     "my-topic",
    Balancer:   &kafka.Hash{},
    RequiredAcks: kafka.RequireAll,
    MaxAttempts: 3,
}
defer writer.Close()

err := writer.WriteMessages(context.Background(), kafka.Message{
    Key:   []byte("key"),
    Value: []byte("value"),
})

```

Go Consumer Template

```

reader := kafka.NewReader(kafka.ReaderConfig{
    Brokers:      []string{"localhost:9092"},
    GroupID:      "my-group",
    Topic:        "my-topic",
    CommitInterval: 0, // Manual commit
})
defer reader.Close()

for {
    msg, err := reader.FetchMessage(context.Background())
    if err != nil {
        break
    }

    // Process message
    process(msg)

    // Commit
    reader.CommitMessages(context.Background(), msg)
}

```

☐ What You'll Learn

After completing this lab, you will be able to:

☐ Understand Kafka Architecture

- Brokers, topics, partitions, offsets
- Replication and fault tolerance
- ZooKeeper/KRaft coordination

☐ Produce Messages

- Different acknowledgment levels
- Batching and compression
- Partitioning strategies
- Error handling

☐ Consume Messages

- Consumer groups and load balancing
- Offset management
- Rebalancing
- Manual vs auto-commit

☐ Implement Patterns

- Point-to-point queues
- Publish-subscribe
- Dead letter queues
- Retry mechanisms
- Request-reply
- Event sourcing

☐ Operate Kafka

- Create and manage topics
- Monitor consumer lag
- Handle failures
- Performance tuning
- Troubleshooting

☐ Write Production Code

- Error handling
- Graceful shutdown
- Idempotency
- Monitoring
- Testing

☐ Next Steps

1. **Complete all 7 labs** in order
2. **Run all Go examples** and understand each pattern
3. **Build a mini-project:**
 - Order processing system
 - Log aggregation pipeline
 - Real-time analytics dashboard
 - Event-driven microservices
4. **Experiment with configurations:**
 - Try different acks levels
 - Test various batch sizes
 - Compare compression algorithms
 - Measure performance impact
5. **Simulate failures:**
 - Stop brokers
 - Network partitions
 - Slow consumers
 - Large messages
6. **Read the complete reference guide** for deep understanding
7. **Join the community:**
 - Ask questions
 - Share learnings
 - Contribute improvements

Support

Getting Help

1. **Check the logs:**

```
make logs
```

2. **Review troubleshooting guide** in the reference documentation
3. **Check common issues** in this README
4. **Search online:**

- Stack Overflow (apache-kafka tag)
- Kafka documentation
- GitHub issues

Useful Links

- ☐ Complete Reference Guide: Artifact above
- ☐ Go Examples: Artifact above
- ☐ Lab Guide: Artifact above
- ☐ Helper Scripts: Artifact above
- ☐ Docker Compose: Artifacts above

☐ License

This learning lab is provided as-is for educational purposes. Feel free to use, modify, and share.

☐ Acknowledgments

Built with:

- Apache Kafka
- segmentio/kafka-go
- Confluent Platform Docker Images
- Kafka UI by Provectus

☐ Summary

This comprehensive Kafka lab provides:

- ☐ Complete Docker setup (single + cluster)
- ☐ 15 working Go examples
- ☐ 7 hands-on lab exercises
- ☐ 20+ helper scripts
- ☐ Full reference documentation
- ☐ CLI cheat sheet
- ☐ Troubleshooting guides
- ☐ Real-world patterns
- ☐ Performance testing tools
- ☐ Production best practices

You now have everything you need to master Apache Kafka!

Start with:

```
make start-single  
make topics  
make test
```

Then open <http://localhost:8080> and explore! ☐

Happy Learning! ☐

For questions or improvements, refer to the complete reference guide and lab exercises provided in the artifacts above.